Sprawozdanie z pracy projektowej
System projektowania schematów transportu miejskiego (*Metro*)

Miron Kwiatkowski, Sara Fijołek, Adrian Sitnicki

26 stycznia 2026

Zakres Aplikacja webowa (frontend) + REST API (backend) + baza danych

Spis treści

1 Opis funkcjonalny systemu **3**
1.1 Użytkownicy i uprawnienia . . . 3
1.2 Zarządzanie schematami . . . 3
1.3 Linie, kategorie i stacje . . . 3
1.4 Relacje i połączenia stacja–stacja . . . 4
1.5 Udostępnianie schematów . . . 4
1.6 Statystyki . . . 4
1.7 Panel administracyjny i logi . . . 4

2 Opis technologiczny **4**
2.1 Frontend . . . 4
2.2 Backend . . . 5
2.3 Konteneryzacja i infrastruktura . . . 5

3 Podziałobowiązków i odpowiedzialności w zespole **5**

4 Instrukcja lokalnego uruchomienia systemu **6**
4.1 Wymagania . . . 6
4.2 Tryb developerski (zalecany lokalnie) . . . 6
4.2.1 Backend + baza (Docker) . . . 6
4.2.2 Frontend (Vite) . . . 6

5 Instrukcja zdalnego uruchomienia systemu (domena + wspólny origin) 6
5.1 Konfiguracja zmiennych środowiskowych 7
5.2 Konfiguracja Cloudflare Tunnel 7
5.3 Start systemu (Docker Compose) 7
5.4 Uwaga: logowanie Google/Facebook 7
5.5 Wnioski projektowe 7
5.6 Linki do repozytoriów github 8

1 Opis funkcjonalny systemu

System *Metro* jest aplikacją webową umożliwiającą tworzenie i zarządzanie schematami sieci transportu miejskiego (np. metro/tramwaj/autobus). Aplikacja pozwala na modelowanie linii, stacji oraz połączeń pomiędzy stacjami, a także na udostępnianie schematów innym użytkownikom. Komunikacja pomiędzy warstwą prezentacji i logiką biznesową jest realizowana przez REST API.

1.1 Użytkownicy i uprawnienia

- Rejestracja konta: `POST /account/register`.
- Logowanie i uzyskanie tokenu JWT: `POST /account/login` (odpowiedź zawiera `access_token`).
- Role i uprawnienia: system wykorzystuje ASP.NET Identity. Przy starcie aplikacji tworzona jest rola `Admin` oraz konto administratora (dane domyślne w kodzie).
- Dostęp do większości zasobów jest chroniony i wymaga nagłówka `Authorization: Bearer <token>`.

1.2 Zarządzanie schematami

- Lista schematów użytkownika: `GET /schemas`.
- Tworzenie schematu: `POST /schemas` (nazwa, opis, miasto, kraj, publiczność).
- Szczegópraca na schemacie obejmuje dodawanie linii, stacji oraz połączeń.

1.3 Linie, kategorie i stacje

- Kategorie linii (np. metro/tramwaj) z nazwą i kolorem: `GET/POST/PUT/DELETE /line-categories`.
- Linie w ramach schematu: `GET /lines/schema/{schemaId}` oraz `POST /lines`.
- Stacje w ramach schematu: `GET /stations/schema/{schemaId}`, dodawanie/edycja-/usuwanie przez `POST /stations`, `PUT /stations/{id}`, `DELETE /stations/{id}`.

Stacja posiada m.in. nazwę, współędne geograficzne oraz pozycję na płótnie edytora (`positionX`, `positionY`), dzięki czemu frontend może rozmieszczać elementy w sposób intuicyjny (*drag&drop*).

1.4 Relacje i połączenia stacja–stacja

- Przypisanie stacji do linii (relacja wiele–wiele): endpointy `/station-lines/...`.
- Odcinki (krawędzie) pomiędzy stacjami na danej linii: `/station-connections/...`. Dane te są wykorzystywane przez frontend do rysowania tras pomiędzy stacjami.

1.5 Udostępnianie schematów

Użytkownik może nadać innej osobie dostęp do schematu (np. tylko podgląd lub edycja). Funkcje obsługuje kontroler `SharedSchema`:

- Udostępnij: `POST /sharedschema/share`.
- Lista udostępnionych przeze mnie: `GET /sharedschema/shared-by-me`.
- Lista udostępnionych mnie: `GET /sharedschema/shared-with-me`.

1.6 Statystyki

System umożliwia prezentację statystyk dotyczących schematu (np. liczba stacji, linii, połączeń). Dane są udostępniane przez `/statistics` (np. `GET /statistics/{schemaId}`) oraz mogą być przeliczane na żądanie.

1.7 Panel administracyjny i logi

Dla użytkowników z rolą `Admin` aplikacja udostępnia funkcje administracyjne (zarządzanie użytkownikami, blokowanie kont, role) oraz podgląd logów serwera.

2 Opis technologiczny

2.1 Frontend

Frontend zostałzrealizowany jako aplikacja SPA w technologii **React 19** z **TypeScript**, z budowaniem przez **Vite**. Nawigacja między widokami jest realizowana przez **React Router**. Logika uwierzytelniania jest enkapsulowana w kontekście (`AuthContext/AuthProvider`).

Warstwa komunikacji z backendem znajduje się w `src/api/api.ts` i realizuje żądania HTTP metodą `fetch`. Adres API jest konfigurowany zmienną `VITE_API_URL`. W trybie developerskim skonfigurowano proxy w `vite.config.ts`, które przekierowuje `/api` na backend i usuwa prefiks `/api`.

2.2 Backend

Backend zostałzrealizowany jako **ASP.NET 8 REST API**. Kluczowe komponenty:

- **Entity Framework Core** + **SQL Server 2022** jako baza danych.
- **ASP.NET Identity** do zarządzania użytkownikami i rolami.
- Autoryzacja przez **JWT** (Bearer token) dla wywołań API.
- (Opcjonalnie) logowanie zewnętrzne: **Google OAuth** i **Facebook Login** (wymaga konfiguracji kluczy w `appsettings`/zmiennych środowiskowych).
- **Swagger/OpenAPI** dla dokumentacji i testowania endpointów.
- **Serilog** do logowania zdarzeń (konsola + pliki w katalogu `Logs`).

W kodzie startowym (`Program.cs`) zastosowano automatyczne wykonywanie migracji przy starcie aplikacji (`db.Database.Migrate()`) oraz inicjalizację roli `Admin` i konta administratora.

2.3 Konteneryzacja i infrastruktura

Projekt wspiera uruchamianie w kontenerach **Docker**. Wariant zdalny (publiczny dostęp) wykorzystuje:

- **Nginx** jako bramę (*gateway*) zapewniającą wspólny origin dla frontendu i API (`/api` jest proxy'owane do backendu z ucięciem prefiksu).
- **Cloudflare Tunnel (cloudflared)** do wystawienia aplikacji na domenie bez konieczności posiadania publicznego adresu IP i przekierowań portów.

3 Podziałobowiązków i odpowiedzialności w zespole

- **Frontend: Miron Kwiatkowski** – implementacja SPA (React/TS), routing, logowanie/rejestracja w UI, widoki zarządzania schematami, edytor schematu (płótno/canvas), panel administratora, warstwa komunikacji z API.
- **Backend: Sara Fijołek** – implementacja REST API (ASP.NET 8), warstwa serwisów i repozytoriów, modele danych i migracje (EF Core), uwierzytelnianie (JWT, Identity), logowanie serwera, integracje (Google/Facebook).
- **Testy: Adrian Sitnicki** – przygotowanie scenariuszy testowych (API/GUI), testy regresji podstawowych funkcji (logowanie, CRUD schematów, stacji, linii), weryfikacja uprawnień (Admin/User) oraz testy integracyjne komunikacji frontend–backend.

4 Instrukcja lokalnego uruchomienia systemu

4.1 Wymagania

- Docker Desktop (Windows/Linux/macOS) lub Docker Engine.
- Node.js (zalecane 18+; w projekcie korzystano z nowoczesnych wersji), npm.

4.2 Tryb developerski (zalecany lokalnie)

Ten wariant wykorzystuje proxy Vite, aby frontend wywoływałAPI pod `/api`, a Vite przekierowywał `/api` na backend i usuwałprefiks.

4.2.1 Backend + baza (Docker)

1. Przejdź do katalogu backendu: `backend/`.
2. Uruchom kontenery bazy i API:

```
cd backend
docker compose up -d --build
```

3. Backend będzie dostępny lokalnie pod `http://localhost:5000` (mapowanie portów jest zdefiniowane w `backend/docker-compose.yml`).

4.2.2 Frontend (Vite)

1. Przejdź do katalogu `frontend/`.
2. Zainstaluj zależności i uruchom serwer developerski:

```
cd frontend
npm install
npm run dev
```

3. Frontend będzie dostępny pod `http://localhost:5173`. Zapytania do `/api/...` zostaną przekazane do backendu przez proxy zdefiniowane w `vite.config.ts`.

5 Instrukcja zdalnego uruchomienia systemu (domena + wspólny origin)

Wariant zdalny z archiwum wykorzystuje **docker-compose** w katalogu głównym oraz **Cloudflare Tunnel**, aby wystawić aplikację pod domeną (np. `mkproj.space`). Frontend

i backend pozostają odseparowane (osobne usługi), natomiast jeden origin jest zapewniany przez bramę Nginx (`gateway`).

5.1 Konfiguracja zmiennych środowiskowych

Utwórz plik `.env` obok `docker-compose.yml` (nie należy przechowywać sekretów w repozytorium). Przykład (wartości przykładowe):

```
MSSQL_SA_PASSWORD=SilneHaslo123!
JWT_KEY=DLUGI_SEKRET_MIN_64_ZNAKI
FB_APP_ID=...
FB_APP_SECRET=...
GOOGLE_CLIENT_ID=...apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=...
```

5.2 Konfiguracja Cloudflare Tunnel

1. Dodaj domenę do Cloudflare i ustaw jej nameservery w panelu rejestratora.
2. Utwórz tunel i podłącz rekordy DNS do tunelu (Cloudflare tworzy rekordy CNAME kierujące do `<uuid>.cfargotunnel.com`).
3. W katalogu `cloudflared/` umieść pliki: `cert.pem`, `<uuid>.json` oraz `config.yml`.

5.3 Start systemu (Docker Compose)

```
docker compose up -d --build
```

Po uruchomieniu aplikacja powinna być dostępna pod `https://<domena>/`. Backend jest osiągalny pod `https://<domena>/api/...` (gateway mapuje `/api` na backend i usuwa prefiks).

5.4 Uwaga: logowanie Google/Facebook

W przypadku logowania zewnętrznego należy poprawnie skonfigurować *redirect URI* w konsolach dostawców OAuth (Google/Meta). Dodatkowo, przy publikacji za pośrednictwem tunelu HTTPS warto dopilnować, aby aplikacja generowała adresy zwrotne z protokołem `https` (np. przez obsługę nagłówków `X-Forwarded-Proto` w backendzie).

5.5 Wnioski projektowe

Projekt spełnił większość założonych celi funkcjonalnych i technologicznych, a integracja frontendu z REST API działa stabilnie w typowych scenariuszach użycia.

W trakcie realizacji okazało się, że warstwa frontendowa była trudniejsza do zaprogramowania niż się początkowo wydawało, szczególnie w obszarze edytora schematów (obsługa interakcji, stan aplikacji, rysowanie na płótnie, walidacja i obsługa błędów API).

Poziom komunikacji w zespole był na relatywnie wysokim poziomie, podział ról był czytelny, a bieżące problemy (np. dopasowanie kontraktów API i modeli DTO) były rozwiązywane na bieżąco.

Niestety nie udało się wprowadzić wszystkich zakładanych funkcjonalności. Projekt możnaby było dużo usprawnić poprzez usprawnienie frontendu o brakujące funkcjonalności i o dokończenie testów, na które zabrakło czasu.

5.6 Linki do repozytoriów github

1. https://github.com/miron-kwiatkowski/MetroFront - frontend
2. https://github.com/SaraFijolek/PipsiII_project/tree/main - backend
3. https://github.com/SaraFijolek/PipsiII_project/tree/test/12-first-tests - testy